fn make_stable_group_by

Michael Shoemate (@Shoeboxam)

November 11, 2026

> This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `make_stable_group_by` in `mod.rs` at commit a105afe (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

- Generic `M` implements `UnboundedMetric`.

Caller-verified

None

Pseudocode

```
def make_stable_group_by(
    input_domain: DslPlanDomain, input_metric: FrameDistance[M], plan: DslPlan
) -> Transformation[DslPlanDomain, DslPlanDomain, FrameDistance[M], FrameDistance[M]]:
    if M.ORDERED:
        raise "Sample-and-aggregate requires an unordered input metric"

    match plan:
        case DslPlan.GroupBy(input, keys, aggs, apply, maintain_order, options):
            pass
        case _:
            raise "Expected group-by in logical plan"

    if apply is not None:
        raise "apply is not currently supported"

    if options != GroupbyOptions.default():
        raise "options is not currently supported"

    t_prior = input.make_stable(input_domain, input_metric)
    middle_domain, middle_metric = t_prior.output_space()

    expr_domain = WildExprDomain(
        columns=middle_domain.series_domains,
        context=Context.RowByRow,
```

[1] See new changes with `git diff a105afe..b137e66b6 rust/src/transformations/make_stable_lazyframe/group_by/mod.rs`

```
    )

    for key in keys:  #
        key.make_stable(expr_domain, L0PInfDistance(middle_metric[0]))

    for agg in aggs:  #
        check_infallible(agg, Resize.Allow)

    schema = Schema.from_columns(middle_domain.series_domains)
    for agg in aggs:  #
        check_permutation_invariance(agg, schema)

    if middle_metric[0].identifier().is_some():  #
        raise "identifier is not currently supported"

    # prepare output domain series
    output_schema = middle_domain.simulate_schema(  #
        lambda lf: lf.group_by(keys).agg(aggs)
    )
    series_domains = [SeriesDomain.new_from_field(f) for f in output_schema]

    # prepare output domain margins
    h_keys = set(keys)

    def output_margin(m: Margin) -> Margin:
        m = m.clone()
        m.by = materialize_grouping_keys(m.by)  #
        m.invariant = None
        return m

    margins = [
        output_margin(m) for m in middle_domain.margins if m.by.is_subset(h_keys)
    ]

    output_domain = FrameDomain.new_with_margins(series_domains, margins)

    def stability_map(d_in: Bounds) -> Bounds:
        #
        contributed_rows = d_in.get_bound(set()).per_group
        #
        contributed_groups = d_in.get_bound(h_keys).num_groups

        influenced_groups = option_min(contributed_rows, contributed_groups)
        if influenced_groups.is_none():
            return (
                "an upper bound on the number of contributed rows or groups is required"
            )

        per_group = influenced_groups.inf_mul(2)  #

        bound = Bound(by=set(), per_group=per_group, num_groups=None)
        return Bounds([bound])  #

    t_group_agg = Transformation.new(
        middle_domain,
        output_domain,
        lambda plan: DslPlan.GroupBy(
            input=plan,
            keys=keys,
            aggs=aggs,
            apply=None,
            maintain_order=maintain_order,
            options=options,
        ),
```

```
        middle_metric,
        middle_metric,
        StabilityMap.new_fallible(stability_map),
    )

    return t_prior >> t_group_agg


# Called only for subsets of validated grouping expressions.
def materialize_grouping_keys(by: set[Expr]) -> set[Expr]:
    return {col(key.meta().output_name()) for key in by}
```

Postcondition

Theorem 1.1. For every setting of the input parameters `(input_domain, input_metric, plan)` to `make_stable_group_by` such that the given preconditions hold,
`make_stable_group_by` raises an error (at compile time or run time) or returns a valid transformation. A valid transformation has the following properties:

1. (Data-independent runtime errors). For every pair of members x and x' in `input_domain`, `invoke`(x) and `invoke`(x') either both return the same error or neither return an error.

2. (Appropriate output domain). For every member x in `input_domain`, `function`(x) is in `output_domain` or raises a data-independent runtime error.

3. (Stability guarantee). For every pair of members x and x' in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for
`output_metric`, if x, x' are `d_in`-close under `input_metric`, `stability_map`(`d_in`) does not raise an error, and `stability_map`(`d_in`) $=$ `d_out`, then `function`(x), `function`(x') are `d_out`-close under `output_metric`.

Appropriate Output Domain. By line 27 the grouping keys are stable row-by-row transformations of the data. By line 30 the aggregates are infallible. This validation also rejects physical row indices and positional Series/range literals, and requires a public, finite scalar quantile parameter in $[0, 1]$: Polars evaluates that parameter against the whole frame, so data-dependent quantiles would not be group-local. Therefore the accepted aggregates do not raise data-dependent errors or access those non-local inputs.

By the postcondition of `DslPlan.simulate_schema`, `series_domains` follows the expected schema of members in the output domain. The output column domains preserve the schema while allowing infallible black-box aggregation.

Only margins whose original grouping expressions are a subset of the grouping keys are preserved. On line 51, each preserved key becomes `col(output_name)`. Polars evaluates a grouping expression on the input and stores its value in the output column with that name. The translated margin therefore partitions output rows by the same labels as the original margin partitions input rows. Each input row contributes to at most one output row, so the upper bounds on group lengths and the number of groups remain valid. Re-evaluating the original expression after aggregation is not generally equivalent and must not inherit the original margin. Among these margins, invariants are discarded.

It has been shown that for every element x in `input_domain`, `function`(x) is in `output_domain` or raises a data-independent runtime exception. □

The stability guarantee doesn't attempt to claim the broadest set of possible bounds on output distances, rather it only claims a simple bound that might be useful for the user. This can be extended in the future, but is sufficient for select queries.

Stability guarantee. We first simplify the problem on line 37 by only considering datasets that differ by rows, not by identifiers.

The function rejects ordered input metrics. For accepted unordered metrics, the output metric equals the input metric, so arbitrary output-group reorderings do not contribute distance. The only output distance bound derived is for the empty grouping set, namely the entire output table.

Under unordered adjacency, zero-distance permutations must preserve the joint distribution of the aggregates. Line 34 enforces a conservative expression allowlist for this purpose. Scalar literals broadcast without depending on row positions; positional Series and range literals do not satisfy that property. Accepted row-wise expressions commute with row permutations. Grouping keys must also have canonical equality representatives; unordered grouping on float-containing keys is rejected because Polars may emit either signed-zero representation. Invariant reducers, such as integer sums, therefore produce the same values. Floating-point min/max are not treated as invariant: Polars' sorted-column fast paths may select bit-distinct tied values. Sensitive reducers, including floating-point sums and means, instead require a trusted uniform random permutation of their input. A generic Polars sort is not treated as canonical: the engine may skip sorting based on existing sorted-state metadata that this analysis does not certify. For uniform permutations, couple corresponding records in the two inputs; the resulting ordered samples agree in distribution, including when only a prefix is retained. Deterministic postprocessing preserves this distributional equality. Independently randomized expressions can be coupled independently. Consequently, unchanged groups can be coupled to have identical aggregates.

We first retrieve optional upper bounds on the number of rows and groups an individual may contribute on lines 62 and 64. Both of these bounds directly correspond to the number of rows an individual may influence in the resulting dataset.

$$\max_{x \sim x'} d_{\text{FrameDistance<M>}}(\texttt{function}(x), \texttt{function}(x')) \tag{1}$$

$$\leq \text{option_min}(\texttt{contributed_rows}, \texttt{contributed_groups}) \cdot 2 \tag{2}$$

Since each influenced row accounts for one addition and one removal, the distance is twice the number of influenced rows. This is reflected on line 73. The resulting bound is returned on line 76, satisfying the postcondition. □